Systemarchitektur

CraftVoice · Architektur und Aufgabenteilung

Patrick Mikosch

1 Systemarchitektur

Es wurde sich auf die folgende Systemarchitektur geeinigt:

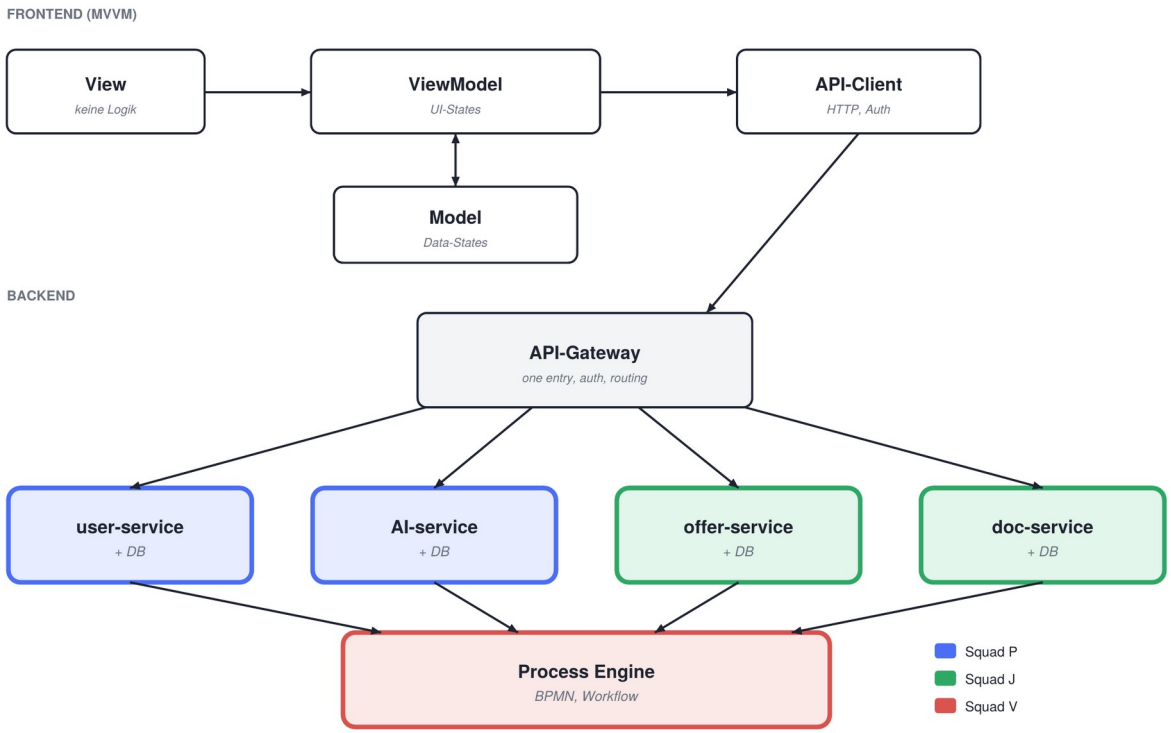


Abbildung 1: Systemarchitektur – Frontend nach MVVM, Gateway, vier Services mit eigener Datenbank und Process Engine

1.1 Der Workflow im Überblick

Die View ist reine Oberfläche (UI) und enthält keine Logik. Das ViewModel enthält die Logik und die Funktionen von Buttons, Eingabefeldern usw. und hält die UI-States, während das Model die Datenwerte wie int, string etc. enthält. Diese Trennung ermöglicht eine saubere Abkopplung vom Backend und von der Process Engine und sorgt damit für Sicherheit und Performance.

Mithilfe eines API-Clients und des API-Gateways kann das ViewModel die Services per Token ansprechen. Das Gateway ist der einzige Einstiegspunkt ins Backend und übernimmt Authentifizierung und Routing. Jeder Service hat seine eigene Datenbank, die entsprechend aktualisiert wird, und spricht je nach Fall die Process Engine an.

Die Process Engine ist das Herzstück des Konzepts. Sie führt die BPMN-Modelle aus und handhabt die verschiedenen Situationen, die in der Applikation stattfinden können.

1.2 Komponenten im Überblick

Schicht	Komponente	Aufgabe	Zuständig
Frontend	**View**	Reine Darstellung (UI), keine Logik	P + J
Frontend	**ViewModel**	Logik und Funktionen von Buttons, Eingabefeldern etc.; hält die UI-States	P + J
Frontend	**Model**	Hält die Daten-States, also Werte wie int, string etc.	P + J
Frontend	**API-Client**	Kommunikation mit dem Backend (HTTP, Auth per Token)	P + J
Backend	**API-Gateway**	Ein Einstiegspunkt ins Backend: Authentifizierung und Routing	–
Backend	**user-service + DB**	Eigener Service mit eigener Datenbank	Squad P
Backend	**AI-service + DB**	Eigener Service mit eigener Datenbank	Squad P
Backend	**offer-service + DB**	Eigener Service mit eigener Datenbank	Squad J
Backend	**doc-service + DB**	Eigener Service mit eigener Datenbank	Squad J
Prozess	**Process Engine**	Führt die BPMN-Modelle aus und steuert den Workflow	Squad V

2 Aufgabenteilung

Durch die Wahl dieser Systemarchitektur hat sich auch die folgende Aufgabenteilung zwischen den drei Squads ergeben:

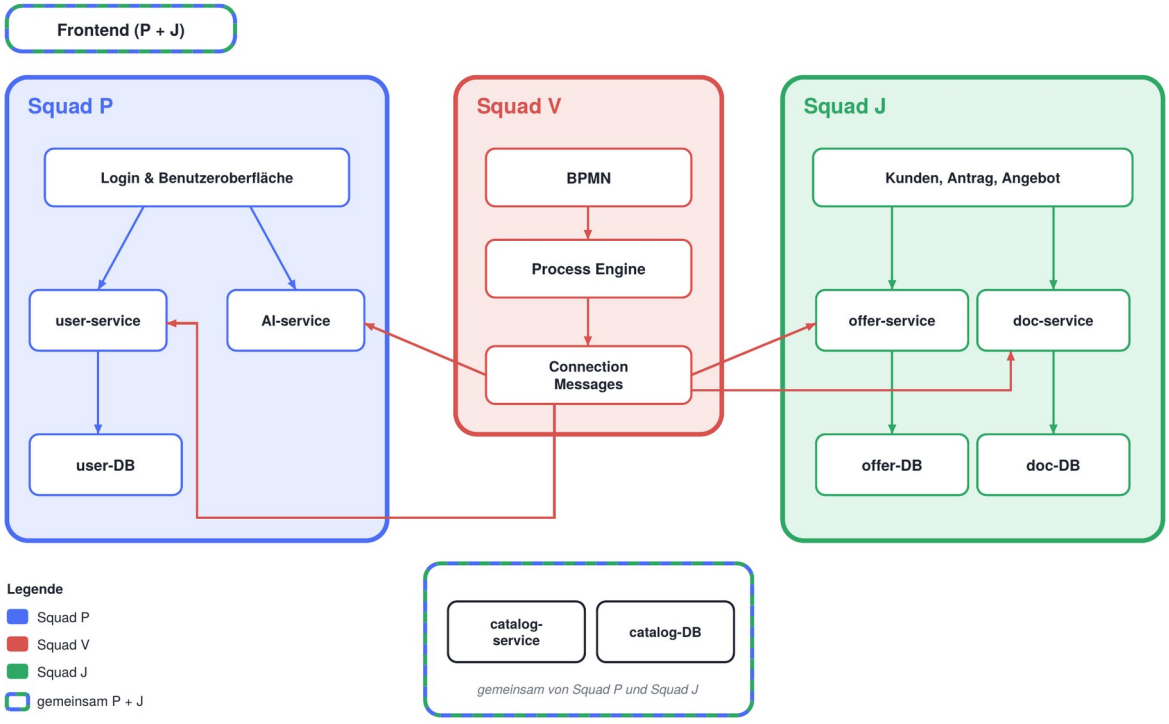


Abbildung 2: Aufgabenteilung der Squads V, P und J (gestrichelt: gemeinsam von Squad P und Squad J)

2.1 Squad V – Process Engine und Prozesse

Verantwortung: Squad V ist für das Herzstück der Anwendung zuständig: die BPMN-Modelle, die Process Engine und die Anbindung der Services an die Engine.

Zu entwickeln

- **BPMN-Modelle:** Die Abläufe der Applikation modellieren, sodass die Engine die verschiedenen Situationen abdecken kann.
- **Process Engine:** Engine aufsetzen und konfigurieren sowie die BPMN-Modelle darin bereitstellen.
- **Connection Messages:** Schnittstelle zwischen der Process Engine und dem user-, AI-, offer- und doc-service. Die Services stoßen je nach Fall die Engine an, die Engine steuert den Ablauf.

Abstimmung: Mit P und J festlegen, welche Nachrichten die einzelnen Services mit der Engine austauschen.

2.2 Squad P – Benutzer, Login und KI

Verantwortung: Squad P kümmert sich um Login und Benutzeroberfläche sowie um alle benutzerbezogenen Funktionen inklusive der KI.

Zu entwickeln

- **Login und Benutzeroberfläche:** Login-Screen und Oberflächen im Frontend nach MVVM (View, ViewModel, Model), angebunden über den API-Client.
- **user-service + user-DB:** Benutzerverwaltung und Login/Authentifizierung als eigener Service mit eigener Datenbank.
- **AI-service:** KI-Funktionen der App als eigener Service mit API (laut Architektur mit eigener Datenbank).
- **Anbindung an die Process Engine:** user-service und AI-service senden und empfangen Connection Messages (gemeinsam mit V).

2.3 Squad J – Kunden, Anträge und Angebote

Verantwortung: Squad J deckt alles ab, was Nutzer mit Kunden tun: Kunden, Anträge, Angebote und die zugehörige Dokumentation.

Zu entwickeln

- **Oberflächen für Kunden, Antrag und Angebot:** Views, ViewModels und Models im Frontend, angebunden über den API-Client.
- **offer-service + offer-DB:** Angebote erstellen, speichern und verwalten, als eigener Service mit eigener Datenbank.
- **doc-service + doc-DB:** Dokumentation und Dokumente zu Kunden, Anträgen und Angeboten, als eigener Service mit eigener Datenbank.
- **Anbindung an die Process Engine:** offer-service und doc-service senden und empfangen Connection Messages (gemeinsam mit V).

2.4 Gemeinsam: Squad P + Squad J

- **Frontend:** Gemeinsames MVVM-Grundgerüst mit View, ViewModel, Model und API-Client. P und J stimmen Struktur, Namensgebung und Anbindung an das Gateway (Token) miteinander ab.

- **catalog-service + catalog-DB:** Gemeinsamer Service mit eigener Datenbank. P und J legen fest, wer welche Teile umsetzt.

2.5 Übersicht

Squad	Zuständig für	Ergebnis
Squad V	**BPMN, Process Engine, Connection Messages**	Lauffähige Process Engine mit den BPMN-Modellen und Nachrichtenanbindung aller vier Services
Squad P	**Login und Benutzeroberfläche, user-service + user-DB, AI-service**	Login, Benutzerverwaltung und KI-Funktionen, über das Gateway erreichbar
Squad J	**Kunden, Antrag, Angebot, offer-service + offer-DB, doc-service + doc-DB**	Oberflächen und Services für Kunden, Anträge, Angebote und Dokumentation
P + J	**Frontend (MVVM), catalog-service + catalog-DB**	Einheitliches Frontend und gemeinsamer Katalog